



نظري

3



12



1440 sp

كلية الهندسة المعلوماتية

السنة الثانية

مفاهيم أساسية في معمارية الحاسوب

د. سيرة أستاذة



RB Informatcs; 21/04/2025

بنیان الحاسوب ا

بسم الله الرحمن الرحيم

تذكرة:

- الـ Architecture: تعني وصف المكونات الأساسية للمعالج والعمليات التي يقوم بها.
 - الـ Organization: تعني كيف يتم تنفيذ هذه المتغيرات فعلياً داخل الجهاز.
- ويكون الـ Architecture هو الجزء الأصعب والأهم لذا عادةً ما نبدأ به، ونعتبره المستوى الأعلى.

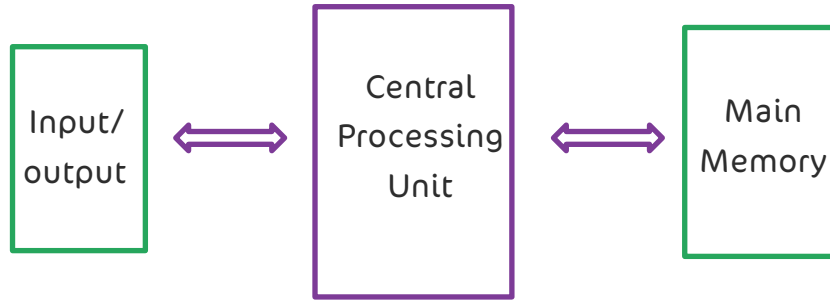
لبدء أي تطبيق / برنامج / مشروع، علينا أولاً القيام بالتحليل، أي فهم المتطلبات (وهو ما نسميه هنا الـ Architecture)، فمثلاً لتصميم معالج: المتطلبات أو العمليات المطلوبة منه: جمع / ضرب / ينقل تعليمات / ينفذ عمليات منطقية... ثم: نفكر بكيفية تنفيذ هذه الأشياء (وهو ما نسميه هنا الـ Organization).

The Von Neumann Model:

تم استكشاف العديد من النماذج المعمارية للحواسيب للأغراض العامة، وكان النموذج الأكثر استخداماً والذي تعتمد عليه جميع الحواسيب الحديثة تقريباً هو النموذج الذي اقترحه John Von Neumann في أواخر 1940s (أربعينيات القرن الماضي)، وهو العالم الذي اقترح الفصل بين المعالج الذي ينفذ العمليات وبين التخزين.

النموذج يتكون من 3 أجزاء رئيسية:

1. وحدة المعالجة المركزية (CPU) Central Processing Unit: هي عقل الحاسوب تقوم بتنفيذ الأوامر ومعالجة البيانات.
2. الذاكرة Memory: تخزن البيانات والتعليمات.
3. أجهزة الإدخال والإخراج I/O.



ما يميز Von Neumann Model:

1. التعليمات والبيانات تخزن معاً في الذاكرة:

أي أن الحاسوب لا يفرق بين التعليمات (أوامر جمع / طباعة...) والبيانات (كالأرقام)، كلها تخزن بنفس المكان.

2. التنفيذ بالتتابع (Sequential Semantics):

بالنسبة للمبرمج، يبدو أن الحاسوب ينفذ الأوامر واحداً تلو الآخر خطوة بخطوة بشكل تسلسلي.

3. الـ CPU تقوم بجلب التعليمات وتنفيذها:

تقرأ الـ CPU كل تعليمة من الذاكرة وتنفذها واحدة تلو الأخرى.

4. The CPU as H/W Interpreter (Hardware Interpreter)

يمكن تخيل الـ CPU كمفسر يقوم بقراءة البرنامج وينفذه.

5. البرنامج نفسه يعامل على أنه بيانات:

أي أن البرنامج عبارة عن بيانات يمكن تغييرها أو تحميلها في الذاكرة مثل أي بيانات أخرى.

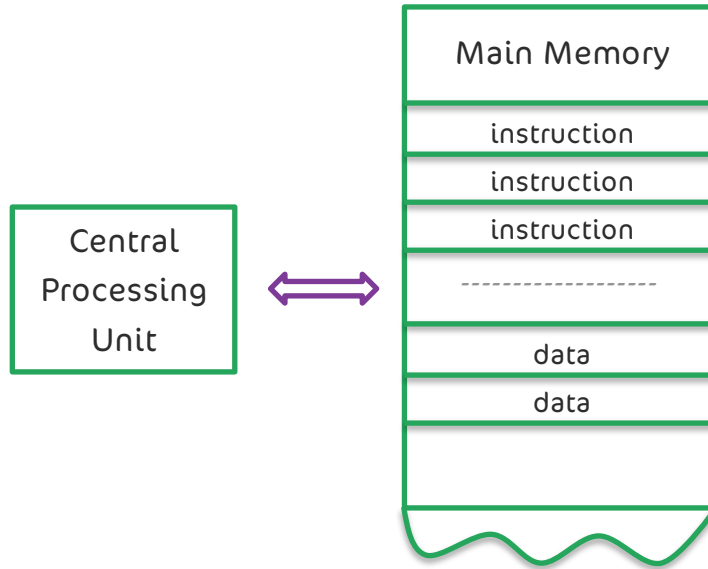
■ ملاحظة:

على الرغم من أن طريقة التنفيذ تكون بشكل تسلسلي Sequential Semantics، إلا أنه يتم استعمال Control Statements أحياناً لحل بعض الإشكالات. مثال عنها:

if / if else / for / while.....

■ انتبه!

كل من التعليمات والبيانات تخزن في الذاكرة ولكن بأقسام مختلفة.



■ Example:

$sum = A + B$

هي تعليمة تحتوي على متغيرات: $sum / A / B$ ، وتخزن هذه المتغيرات بقسم الـ Data (البيانات) في الذاكرة وليس بقسم التعليمات.

Fetch and Execute Cycles:

هذا هو الأساس في عمل الـ CPU، وتكرار هذا الدوران هو ما يجعل البرنامج يعمل.

1. Fetch (جلب):
الـ CPU تجلب التعليمة من الذاكرة وتضعها في سجل خاص يسمى (Program Counter (PC)، وهو مكان يخزن فيه عنوان التعليمة الحالية.
2. تحديد العملية Decode Opcode:
تفكك التعليمة، وعن طريق جزء Opcode يتم تحديد نوع العملية المطلوبة (جمع / نقل / قفز.....).
3. تنفيذ العملية.
4. جلب التعليمة التالية: تعاد الدورة من جديد.

There is no elevator to success, You have to take the Stairs ♥

Instruction Set Architecture: (ISA)

هي مجموعة تعليمات لغة التجميع (Assembly) التي تشكل صلة وصل بين الـ Software والـ Hardware. يجب أن تحتوي على تعليمات كافية لترجمة أي برنامج بشكل فعال حتى يتمكن الحاسوب من معالجته، فمثلاً للقيام بعملية جمع:

- الـ Software يطلب عملية الجمع: `int sum=a+b;`
 - الـ Assembly: يترجم الطلب إلى تعليمات بسيطة.
 - الـ ISA يحول هذه التعليمات إلى صيغ ثنائية مفهومة للمعالج.
 - الـ Hardware ينفذ هذه التعليمات باستخدام دارات مثل Full Adder.
- ولذلك نقول أن الـ ISA صلة وصل بين الـ Software والـ Hardware.

■ شرح أرسيزي:

الفرق بين الـ ISA والـ Assembly:

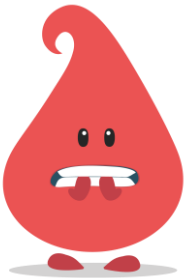
- الـ ISA: هي مجموعة التعليمات التي يفهمها المعالج، مثل القوانين أو اللغة الرسمية التي يستخدمها المعالج لتنفيذ الأوامر.
- الـ Assembly: هي تمثيل نصي لتعليمات الـ ISA يكتبه المبرمج، كل أمر في لغة الـ Assembly يترجم مباشرة إلى تعليمة من الـ ISA.

مثال:

في الـ ISA: توجد تعليمة لنقل البيانات من مكان لمكان.
في الـ Assembly: تكتب هذه التعليمة بشكل مفهوم للمبرمج (Mov R1,R2).

اختلاف الـ ISA من معالج لآخر:

كل معالج له لغة خاصة به (ISA) والـ Assembly تعتمد على هذه اللغة.
كل معالج مبني على نوع معين من الـ ISA (مثل x86 أو ARM).
فالتعليمات التي يفهمها معالج Intel ليست نفسها التي يفهمها معالج ARM.



■ إطار ISA (ما الذي يشملها):

1. Data types (int, float, double..):

يجب أن تكون البيانات والتعليمات معرفة على مستوى المعالج (مستوى الـ Hardware) أولاً حتى نستطيع استعمالها على مستوى الـ High-level language.

مثال: كيف يعرف المعالج الـ int؟ أي يحجز لها 4byte في الذاكرة.

2. Data transfer modes (instructions) between Registers, Memory and I/O.

3. Registers:

كم عددها وحجمها وأسماءها وما وظيفتها.

4. Addressing modes:

كيف نحدد أماكن البيانات في الذاكرة، وفيها أكثر من نوع منها:

- عنونة مباشرة: ما يعطى في التعليمة هو عنوان البيانات مباشرةً، ويتم ذلك بخطوة واحدة.
- عنونة غير مباشرة: ما يعطى في التعليمة هو عنوان يحتوي على عنوان البيانات، ويتم ذلك بخطوتين (قراءة العنوان ثم البيانات).

5. Memory Architecture:

أي شكلها، كيف يتم بناءها وتوزيعها، وطريقة تركيبها.

6. Interrupts:

أي مقاطعة سير المعلومات، كيف يتفاعل المعالج مع الأحداث الخارجية (سواء أخطاء أو غيرها)، فتحدث المقاطعة مثلاً عند محاولة القسمة على صفر.

مثال آخر: عند تشغيل عدة تطبيقات بنفس الوقت (مثلاً: متصفح، مشغل موسيقى، Word...)،

نظام التشغيل يستخدم المقاطعات لينقل السيطرة بين برنامج وآخر بدقة.

مثال آخر: في حالة الوصول غير القانوني للذاكرة (Illegal memory access)، إذا حاول برنامج أن يقرأ أو يكتب على مكان غير مسموح له في الذاكرة.

7. Exception handling

8. Illegal memory access.

9. I/O to external devices.

أخيراً يتم تمثيل البيانات بشكل ثنائي (Binary).

أنواع المعالجات من حيث التعليمات:

■ CISC (Complex Instruction Set Computer):

- يحتوي على عدد كبير من التعليمات (أحياناً مئات).
 - بعض التعليمات معقدة وتحتاج عدة دورات زمنية (cycles) للتنفيذ.
 - الهدف: تقليل عدد التعليمات التي يكتبها المبرمج، لكن على حساب السرعة.
- أمثلة: IBM370, Intel x86, Pentium, i6.

■ RISC (Reduced Instruction Set Computer):

- يحتوي على عدد قليل من التعليمات (عادةً حوالي 32).
 - التعليمات بسيطة وسهلة، وكل واحدة تنفذ في دورة زمنية واحدة تقريباً.
 - هذا يسمح باستخدام تقنية الـ pipelining بكفاءة (تنفيذ عدة تعليمات بتداخل).
- أمثلة: ARM, SPARC, MIPS.
- والمثال الذي ندرسه عادةً هو الـ MIPS.

وهناك أنواع أخرى مثل:

■ VLIW (Very Long Instruction Word)

المعالج ينفذ عدة تعليمات في وقت واحد لأنها مكتوبة معاً في "تعليلة طويلة جداً".

■ Superscalar.

وهما معروفان بالـ Architecture الأعقد.

Operands of Computer Hardware:

الـ Operands هي المعاملات، الـ Operations هي العمليات.

■ Example:

$Sum = A + B$

معاملات هذه العملية هي: sum, A, B



في لغات البرمجة العادية، يمكنك إجراء العمليات الحسابية على أي متغير، لكن في MIPS، لا يمكن إجراء العمليات الحسابية إلا على بيانات موجودة في أماكن خاصة داخل المعالج تسمى السجلات (Registers).

■ ما هو السجل؟

مكان صغير وسريع داخل المعالج لتخزين البيانات، في MIPS، كل سجل حجمه 32 بت.

■ ماهي الـ "Processor" word؟ (كلمة Processor)

هي وحدة الوصول الأساسية أو الطبيعية (natural unit of access) التي يتعامل معها المعالج في MIPS أيضاً تساوي 32 بت، نفس حجم السجل. (أوائل المعالجات بدأت بعرض بـ 4 بت).

■ باختصار:

■ العمليات الحسابية في MIPS تتم فقط على القيم داخل السجلات، وكل سجل (وكل كلمة في المعالج) حجمه 32 بت، فنأتي بالمعاملات من الـ RAM ونخزنها بالـ Registers.

■ لذلك MIPS تحتاج إلى تعليمات خاصة تقوم بنقل البيانات بين الذاكرة والسجلات، هذه التعليمات تسمى Data Transfer Instructions.

■ العنوان (Address):

رقم يستخدم لتحديد مكان البيانات داخل الذاكرة.

■ ملاحظة:

3	100
2	10
1	101
0	1

Address

Data

العناوين لا تخزن داخل الذاكرة نفسها، بل هي تستخدم لتحديد مواقع البيانات داخل الذاكرة، بمعنى آخر، العنوان هو رقم خارجي يشير إلى موقع معين في الذاكرة، وليس جزءاً من البيانات المخزنة.

■ ملاحظة:

عند التعامل مع المحارف characters يستخدم المعالج ترميز ASCII وهو نظام يمثل كل حرف باستخدام 8 بت، هذا يعني أن ASCII يمكنه تمثيل 256 حرفاً مختلفاً تتراوح قيمتهم من 0 إلى 255.

MIPS Processor Instruction Set (RISC)

■ ملاحظة:
انتبه!

معالج MIPS الذي نتحدث عنه هو اختصار لـ Microprocessor without Interlocked pipeline Stages وهو اسم معمارية معالجات تعتمد على تعليمات بسيطة وسريعة (RISC) قد تدل MIPS أحياناً على معاني أخرى، وحينها تكون اختصار لـ Million Instructions per second وهي وحدة لقياس أداء المعالج.

■ كل تعليمة (instruction) تنفذ وظيفة بسيطة واحدة.

■ كل تعليمة تكون مكتوبة بشكل منتظم وثابت تتكون من كلمة واحدة تحتوي على:

■ operation code (opcode) وهو نوع العملية.

■ Arguments المعطيات، مثل أسماء السجلات التي تحتوي على البيانات.

■ نحاول دائماً تقليل استخدام الذاكرة لأنها أبطأ، لذا نستخدم السجلات Registers قدر الامكان لأنها أسرع.

في RISC لتقليل استخدام الذاكرة تم التركيز على استخدام عدد كبير من السجلات بدلاً من الاعتماد على تحميل البيانات من الذاكرة في كل عملية، لأن الوصول إلى السجل أسرع بكثير من الوصول إلى الذاكرة، أما CISC فيها عدد أقل من السجلات.

■ التعليمات هي أوامر يرسلها المبرمج إلى وحدة المعالجة CPU.

■ Example:

Add \$s1, \$s2, \$s3 # \$s1 = \$s2 + \$s3

هذه التعليمية تطلب من المعالج أن يقوم بعملية جمع.

تجمع محتوى السجل \$s2 مع محتوى \$s3 وتخزن النتائج في \$s1.

أنواع التعليمات في معالج MIPS:

1. Load / Store:

التحميل Load وهي read (تقرأ البيانات).

التخزين store وهي write (تكتب البيانات).

ملاحظة:

لدينا معمارية تسمى بـ load / store Architecture ومعالج الـ MIPS هو منها، وفيها التعليمات التي تتعامل مع الذاكرة هي فقط تعليمات store / load، بمعنى آخر، لا يمكن الوصول إلى الذاكرة أو عنوانها إلا من خلال هذه التعليمات فقط.

2. Computational

تشمل التعليمات التي تجري عمليات حسابية (جمع / طرح / ضرب / قسمة / ...).

3. Jump and Branch (قفز):

Jump (قفز غير مشروط).

يعني أن البرنامج ينتقل مباشرة إلى عنوان معين بدون التحقق من أي شرط (مثل تعليمة go to في لغة باسكال)

Branch (قفز مشروط).

يعني أن البرنامج ينتقل إلى مكان معين فقط إذا تحقق شرط معين.

4. Floating point (Coprocesor):

في MIPS يوجد نوعين من السجلات، 32 سجل للـ integer و 32 سجل للـ float، وكل سجل هو 32 بت. الـ floating point مخصصة للتعامل مع الأعداد العشرية.

5. Memory Management.

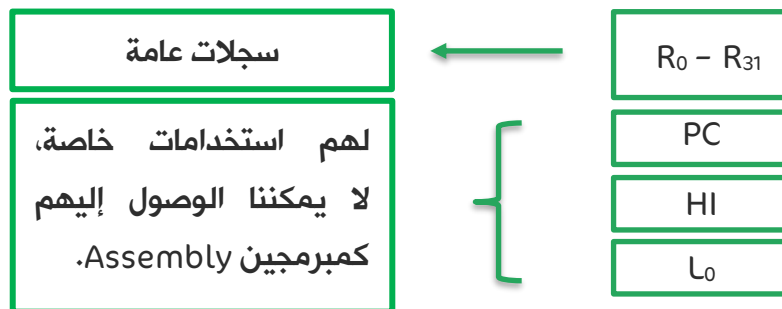
■ ملاحظة:

■ تختلف أسماء السجلات بحسب نوع المعالج.

في MIPS: \$zero, \$at, ...

في Intel: EAX, EBX,

■ \$0 لا نستطيع الكتابة عليه فقط نقرأ منه.



■ program Counter: PC

يستخدم لتحديد عنوان التعليمة التالية التي يجب على المعالج تنفيذها.

■ مثال للتبسيط:

تخيل أنك شغلت برنامج وهذا البرنامج تم تحميله في الذاكرة RAM، كيف يعمل سجل PC؟ النظام يضع أول عنوان للتعلمية في سجل PC ويتم تنفيذها ثم يزيد PC أوتوماتيكياً لينتقل إلى التعليمة التالية.

■ MIPS Instruction Set:

كل تعليمات MIPS تمثل 32 بت، وتتضمن كل تعليمة حقولاً متنوعة، كل حقل له وظيفة محددة:

■ A 6-bit operation or "OPCODE"

- يعبر عن العملية التي يقوم بها المعالج (+ / * / - / XOR /).
- يمكن أن تمثل 64 عملية مختلفة (لأن $2^6 = 64$).
- أحياناً يستخدم "escape code" العمليات أكثر تعقيداً.

■ Several 5-bit OPERAND fields

- تستخدم لتحديد السجلات (registers) المشاركة في العملية.
- MIPS يحتوي على 32 سجل ولذلك احتجنا 5 بت للإشارة إليهم ($2^5 = 32$).

■ Embedded Constants ("immediate" values):

Example:

$Sum = a + 15$

Variable immediate

قد يكون 26 bit / 5 bit / 16 bit
وأحياناً تكون هذه القيم signed وأحياناً بدون إشارة.

■ أنواع التعليمات في MIPS:

لدينا ثلاث أنواع:

1. R-type (Register-type):

- تستخدم فقط سجلات (Registers).
- لا تحتوي على أرقام ثابتة (immediate).
- 3 register operands (2 Sources, 1 destination).

		المعامل الأول	المعامل الثاني	النتيجة		
R	OP	rs	rt	rd	shamt	funct
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

■ ملاحظة:

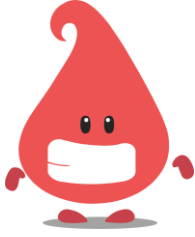
ترتيب المعاملات في التعليمة ضمن الجدول (الملونة بالأخضر) يختلف عن التعليمة ضمن الكود في MIPS، حيث يكون ضمن الكود:

rd, rs, rt

مثل:

add \$t2, \$t1, \$t0

rd rs rt



الـ rs والـ rt والـ rd هم سجلات لذا احتجنا 5 bits لعنونتهم.

- register source :rs (المعامل الأول) input.
- register temporary :rt (المعامل الثاني) input.
- register distention :rd (الناتج) output.
- Shift amount (shamt): مقدار الإزاحة.
- function: تحدد نوع العملية (جمع / طرح /).
- op: يحدد نوع التعليمة (J-type / I-type / R-type /).

2. I-type (Immediate type):

تستخدم سجلين وقيمة ثابتة (immediate) من 16 بت، لذا تستخدم للعمليات مع أرقام.

	المعامل الأول	الناتج	القيمة الثابتة
I	OP	RT	Immediate
	6 bits	5 bits	16 bits

■ ملاحظة أرسيزية:

الـ opcode يمكن أن يحدد:

العملية نفسها (جمع / طرح / ...) أو نوع التعليمة فقط، اذا كانت من نوع R-type في هذه الحالة نحتاج للـ funct لاحقاً لتحديد نوع العملية.

3. J-type (القفز):

- لا تستخدم سجلات.
- تحتوي فقط على عنوان (Address) من 26 بت.

J	OP	Address
	6 bits	26 bits

■ ملاحظة:

الـ rt (register temporary) سميت كذلك لأنها تختلف حسب نوع التعليمة، فمثلاً:
في الـ R-type كانت input.
وفي الـ I-type أصبحت output.

■ فكر؟!

إذا كان عرض ناقل العناوين (Address Bus) هو 32 بت فهذا يعني أن المعالج يستطيع توليد 2^{32} عنواناً مختلفاً، فإن أقصى حجم ذاكرة يمكن الوصول إليه هو $2^{32} = 4,294,967,296 = 4 \text{ GB}$
أي أن أقصى ذاكرة RAM يمكن للمعالج التعامل معها مباشرة هي 4GB.



..انتهت المحاضرة..